

Übung 4

Aufgabe 1

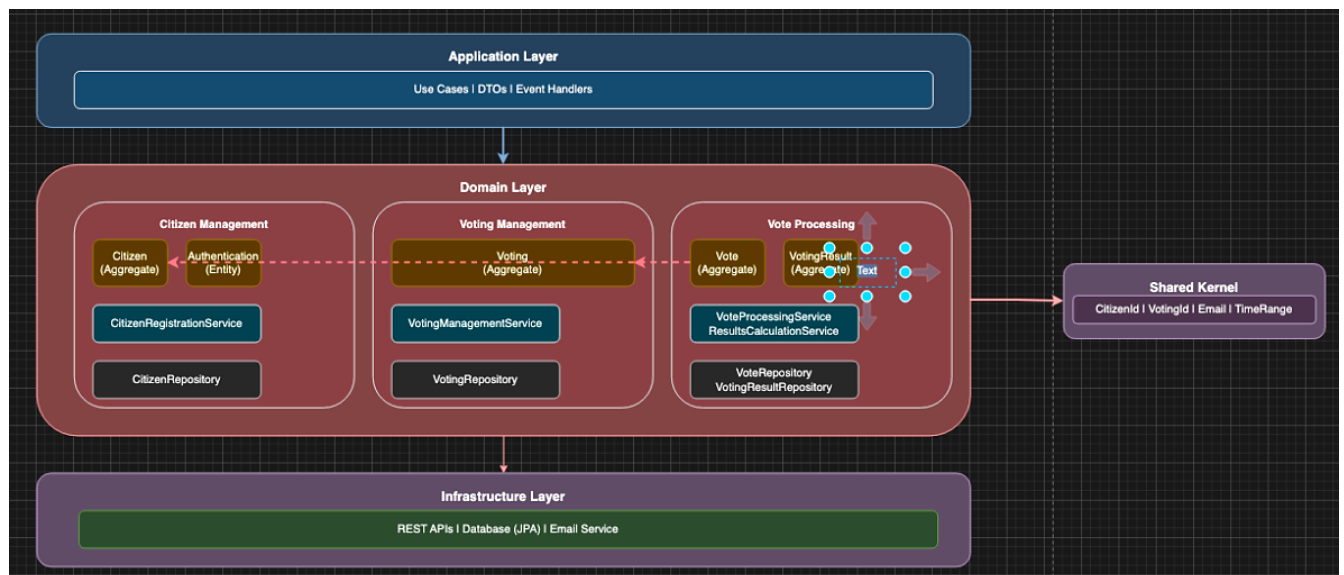
Review Ihrer Implementierungsstrategie: Überprüfen Sie die Implementierungsstrategie aus dem letzten Übungsblatt. Nutzen Sie eine LLM, um (a) Ihre Implementierungsstrategie zu analysieren und potenzielle Schwachstellen zu identifizieren; (b) Vorschläge für eine bessere Strukturierung Ihrer Domain-Events zu erhalten; (c) Die Entwicklungsreihenfolge zu optimieren.

Vorgehen: Präsentieren Sie dem LLM Ihre bisherige Strategie und Bounded Contexts; lassen Sie sich mindestens 3 konkrete Verbesserungsvorschläge geben; bewerten Sie kritisch, welche Vorschläge sinnvoll sind und welche nicht; Passen Sie Ihre Strategie entsprechend an

Ziel/Output: Die wichtigsten Domain-Events (mind. 2) Ihres Projekts sowie ihre sinnvoll angepasste Entwicklungsstrategie werden nun festgehalten. Welche LLM-Vorschläge haben Sie übernommen/abgelehnt und warum? (ca. 5-6 Sätze)

Wir haben uns beim LLM für ChatGPT entschieden, da ChatGPT gut geeignet ist für Review und Verbesserung Implementierungsstrategien sowie Testgenerierung, und Refactoring- und Codequalitäts-Überprüfungen. Außerdem bietet ChatGPT sehr verständliche Erklärungen, wie in unserem Fall bezüglich DDD, TDD und Java-Features und baut Antworten sehr strukturiert auf. Es ist natürlich verständlich, dass ChatGPT auch Fehler machen kann. Daher verwenden wir die KI nur als „Partner“ und überprüfen stets die Antworten.

Unsere Implementierungsstrategie sieht wie folgt aus:



Die KI hat unsere Implementierungsstrategie auf folgende Punkte überprüfen müssen:

- (a) Ihre Implementierungsstrategie zu analysieren und potenzielle Schwachstellen zu identifizieren

Die KI fasst unsere Implementierungsstrategie wie folgt zusammen: „drei Schichten in typischem DDD-Stil:

- **Application Layer:** Use-Cases, DTOs, Event Handlers — orchestriert Domänenoperationen, spricht Repositories/Services an.
- **Domain Layer:** Drei Bounded Contexts (Citizen Management, Voting Management, Vote Processing) mit Aggregaten, Domain-Services und Repositories (Interfaces).
- **Infrastructure Layer:** konkrete Adapter (REST, DB, Email).
- **Shared Kernel:** gemeinsame primitive Typen/IDs (CitizenID, VotingID, Email, TimeRange).

Das ist ein sauberer Start — Bounded Contexts sind klar getrennt und du hast Service/Repository-Abstraktionen vorgesehen.“

ChatGPT analysiert die Implementierungsstrategie und listet potenzielle Schwachstellen auf:

1. **„Authentication als Value im Domain Layer vs. Infra**
 - Problem: Wenn Auth-Mechanik (Tokens, Signaturen) als Domain-Value behandelt wird, vermischst du fachliche Regeln und technische Implementierung. Das erschwert Tests und Austausch der Auth-Implementierung.
 2. **Unklare Transaction Boundaries / Invariant Enforcement**
 - Problem: Die Kerninvariant „max. 1 Stimme pro Bürger pro Abstimmung“ ist nicht spezifiziert, wie sie erzwingbar ist (synchron DB-Constraint vs. asynchrone Deduplication/Saga). Ohne Entscheidung drohen Race Conditions oder Overengineering.
 3. **VotingResult als Aggregate statt Projection**
 - Problem: Wenn VotingResult schreibbar bleibt (Aggregate), vermischst du berechnete Daten mit Autoritativen Writes — das macht Skalierung & Reconciliation schwieriger. Besser: Projection/Read Model.
 4. **Events & Audit nicht ausreichend spezifiziert**
 - Problem: Es fehlen Regeln zu Pseudonymisierung/PII in Events, Idempotency, Event-Sourcing vs. simple Event-Publishing. Das kann Datenschutzverletzungen und nicht-deterministisches Verhalten in Verarbeitungspipeline verursachen.
 5. **Testbarkeit / Infra-Abhängigkeiten zu früh**
 - Problem: Wenn Repositories/Adapters nicht als Ports/Interfaces früh vorgesehen sind, wirst du beim TDD-Start schnell auf konkrete Infra (DB) angewiesen — langsame Tests, schweres Refactoring.“
- (b) Vorschläge für eine bessere Strukturierung Ihrer Domain-Events zu erhalten
- Folgende Verbesserungsvorschläge schlägt ChatGPT vor:
1. **„Auth als Port/Adapter: Domain kennt nur VerificationStatus / pseudonymToken**
 - Nutzen: Domain bleibt testbar; Auth-Implementierung tauschbar; PII bleibt in Infra.
 - Kosten: Mehr Interfaces/Boilerplate, aber klarer DDD-Boundary.
 2. **Trennung: Vote als eigenes Aggregate (append-only), VotingResult als Projection (CQRS)**
 - Nutzen: ermöglicht Pseudonymisierung, append-only Speicherung, einfache Replays für Audit; skalierbar.
 - Kosten: Mehraufwand für Event Handler & Projections; zusätzliche Komponenten (worker).

3. Definiere dedizierte Transaction-Strategy pro Invariant

- Nutzen: Vermeidet Race-Conditions (z. B. synchron für MVP, asynchron für Scale).
- Kosten: Entscheidungsaufwand; später Refactor möglich.“

(c) Die Entwicklungsreihenfolge zu optimieren

Ziel: schnelle Feedback-Loops, kleine Schritte, leicht testbar — starte bottom-up:

1. Value Objects + Validatoren (TDD)

- Email, Name, TimeRange, PseudonymToken — schreibe Unit-Tests (Happy/Edge/Negative).
- *Warum*: klein, determiniert, schnelle Tests; Grundlage aller Aggregates.

2. InMemory Repositories & Test Infrastructure

- Implementiere InMemory/Stub Repos für Citizen/Voting/Vote + simple in-memory EventBus.
- *Warum*: ermöglicht TDD ohne DB-Infra.

3. Citizen Aggregate + CitizenRegistrationService (TDD)

- Tests: Valid registration, duplicate email, markVerified. AuthPort als Mock.
- *Warum*: baut user model, needed for auth flows.

4. Voting Aggregate + VotingManagementService (TDD)

- Tests: create voting, invalid TimeRange, open/close, addOption rules. Emit PollOpened.
- *Warum*: core domain; other flows depend on it.

5. Vote Aggregate + VoteRepository existsBy... behavior (TDD)

- Implement append-only Vote model, validation of option existence (via VotingRepository mock). Tests: submit, invalid option.
- *Warum*: core for cast flow.

6. CastVoteUseCase orchestration (sync variant) + tests (integration with InMemory repos + AuthPort stub)

- Test flow: auth -> check voting open -> dedupe check -> save vote -> emit VoteSubmitted.
- *Warum*: provides deterministic behavior for MVP; easy to test.

7. Event Handlers & Projections (VotingResult) + idempotency tests

- Implement handler for VoteSubmitted → update VotingResult projection. Tests: repeated event handling no double-count.
- *Warum*: validates event pipeline and read model correctness.

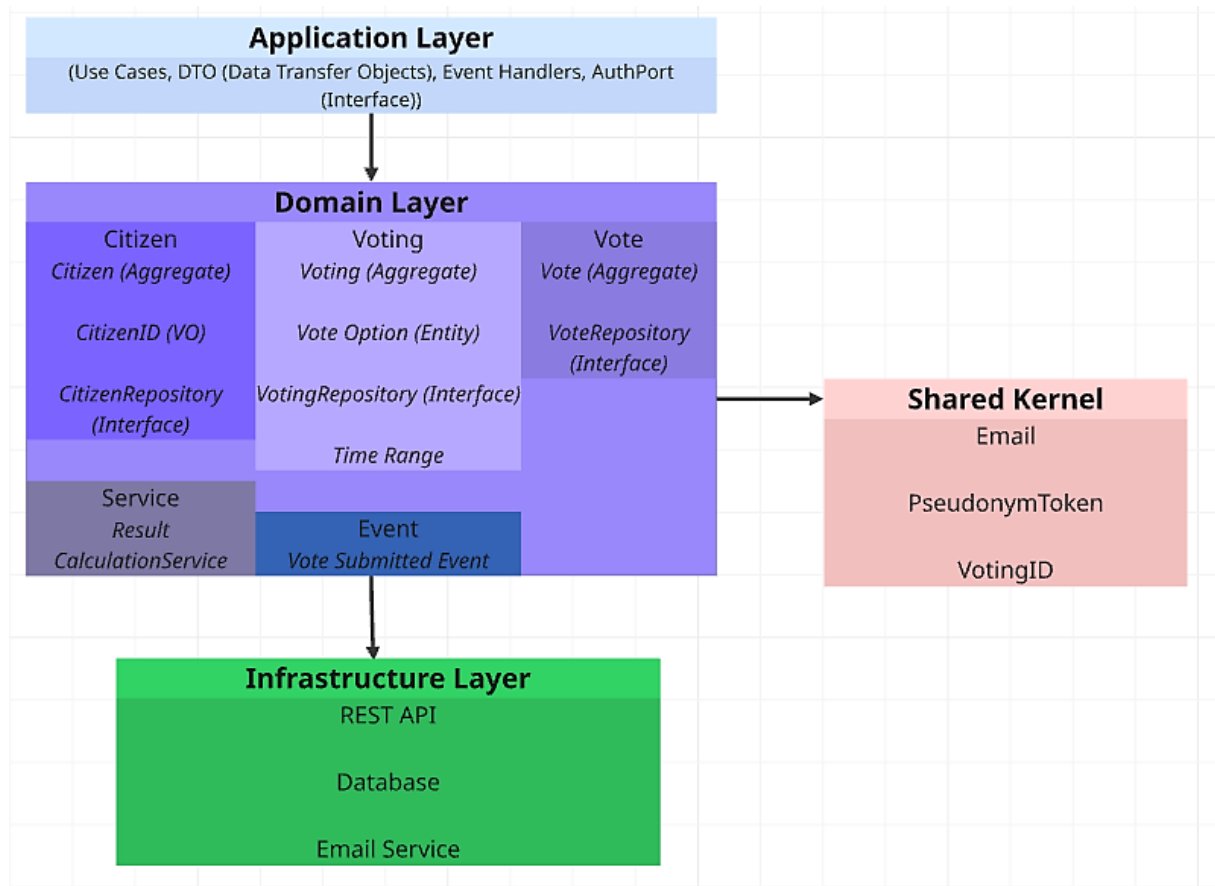
8. Refactor: Replace InMemory with JPA/H2 (integration tests) / add real AuthAdapter stub

- Run integration tests, fix infra issues.
- *Warum*: ensures real infra compatibility.

9. Optional: Asynchronize flow (scale) — enqueue events, worker for dedupe/sagas, add reconciliation tests

- Move to event queue, create background processor for dedupe, update projections via events.
- *Warum*: scale readiness; complex but optional for homework.

Schrittweise wurde die Implementierungsstrategie, die Vorschläge, Schwächen und die jeweiligen Bauteile eines Domänenmodell nach DDD und mithilfe der KI konnte letztendlich eine neue Implementierungsstrategie entwickelt werden:



Die KI hat beispielsweise vorgeschlagen im Domain Layer Value Objects zu verankern und hierin Events wie Email, Time Range und Pseudonym Token zu „packen“. Diesen Vorschlag haben wir nicht angenommen und stattdessen unseren Shared Kernel behalten. Im Shared Kernel sind die gemeinsamen wiederverwendbaren Domänen-Bausteine enthalten. Time Range gehört daher nicht in Value Objects oder Shared Kernel. Time Range ist abhängig von der jeweiligen Abstimmung und ist daher im Domain Layer unter Voting. Weiter noch waren sehr konkrete und präzise Vorschläge für den Infrastructure Layer (seitens der KI) vorgeschlagen worden, die wir nicht in unsere neue Implementierungsstrategie übernehmen wollten, da wir eine grobe Übersicht im Infrastructure Layer behalten wollten.

Die wichtigsten Events haben wir festgelegt:

- Abstimmung starten (mit Option, damit überhaupt eine Stimme abgegeben werden kann und damit aufgrund der abgegebenen Stimmen zu einem Ergebnis kommen kann).
- Stimme abgeben können (damit Bürger sich nicht nur registrieren können, sondern auch abstimmen könne, um Veränderungen vornehmen bzw. beeinflussen zu können)

Diese wichtigsten Events wurden ChatGPT vorgeschlagen und das ist die Antwort:

„Ja — die beiden Events, die du ausgewählt hast, **passen sehr gut als zentrale Domain-Events**. Ich erkläre, warum und worauf du achten solltest:

1. **Event „Abstimmung gestartet“ (VotingStartedEvent)**

Warum wichtig:

- Es markiert den **Beginn der Abstimmung** – ohne diesen Event kann niemand abstimmen.
- Triggert oft **weitere Prozesse**, z. B.:
 - Benachrichtigung an Bürger (Email / Push)
 - Aktivierung von Abstimmungs-Logik in der Application Layer
 - Event-Sourcing / Audit Trails

Fokus: Fachliche Tatsache „Es gibt jetzt eine laufende Abstimmung“ – **keine technischen Details** wie DB oder REST.

2. Event „Stimme abgegeben“ (VoteSubmittedEvent)

Warum wichtig:

- Kern der Domäne: **Bürger nehmen Einfluss auf Abstimmungsergebnisse.**
- Wird für mehrere Zwecke gebraucht:
 - Berechnung der Ergebnisse (ResultCalculationService)
 - Event-basierte Updates (Projektionen, Dashboards)
 - Audit / Nachvollziehbarkeit

Fokus: Fachliches Ereignis, **wer hat wann für welche Option gestimmt** – anonymisiert über PseudonymToken.

3. Warum diese Events gut sind

- Sie **bilden den Kern der Domäne ab**: ohne Abstimmung gibt es keine Stimme, ohne Stimme keine Ergebnisse.
- Sie eignen sich hervorragend für **Event-Publishing**, also lose Kopplung zwischen Domain und Application/Infrastructure.
- Sie können leicht in **TDD-Tests** als Trigger für UseCases verwendet werden.

4. Optional / Ergänzend

Man könnte noch weitere Events definieren, z. B.:

- VotingClosedEvent → Abstimmung ist beendet
- CitizenRegisteredEvent → neuer Bürger registriert

Aber für die **zentrale Fachlogik deines eVote-Systems** sind deine beiden Events **genau richtig**.



Kurz gesagt:

„VotingStarted“ markiert den Beginn, „VoteSubmitted“ das Kernverhalten der Domäne – zusammen bilden sie das Herz der Business-Logik.“